

DEREXI: POLICY PILOT

Hashing & Integrity

PBKDF2-HMAC-SHA256 • Salt • Password Verification

OBJECTIVE

This lab demonstrates how cryptographic hashing protects credentials and how salts make identical passwords produce different stored values. The implementation uses PBKDF2-HMAC-SHA256 with a random salt, then verifies both a correct and incorrect password without storing a real credential.

```

97 # Remember, here is the code that runs on a classroom copy; the real
98 # production value would be much higher.
99 iterations = 100_000
100
101 # A clearly fake demo password. NEVER put a real password in a script.
102 fake_password = b"DeRexi-Demo-Password-2026"
103
104 # derived_key = PBKDF2-HMAC-SHA256(password, salt, iterations)
105 derived_key = hashlib.pbkdf2_hmac(
106     "sha256", # hash algorithm
107     fake_password, # the password (never logged or stored)
108     salt, # the random per-user salt
109     iterations, # how many rounds
110     dklen=32, # output length in bytes (32 bytes = 256 bits)
111 )
112
113 # Show that the same password with a DIFFERENT salt produces a
114 # completely different hash -- this is what defeats rainbow tables.
115 different_salt = secrets.token_bytes(16)
116 different_hash = hashlib.pbkdf2_hmac(
117     "sha256", fake_password, different_salt, iterations, dklen=32
118 )
119
120 print("Demonstration password (fake):", fake_password.decode())
121 print()
122 print(f"Random salt (hex): {different_salt.hex()}")

```

Figure 1. PBKDF2-HMAC-SHA256 derives a salted password hash.

01 Random salt

`secrets.token_bytes(16)` generates an unpredictable per-user salt before hashing.

02 Key derivation

`hashlib.pbkdf2_hmac()` repeatedly applies SHA-256 to derive a 32-byte value.

03 Safe comparison

`hmac.compare_digest()` checks the recomputed digest for correct and incorrect passwords.

WHY THIS MATTERS

Hashing is one-way: the application stores a derived fingerprint rather than a usable password. A unique salt prevents equal passwords from producing equal hashes and reduces the usefulness of precomputed rainbow-table attacks.

LAB RESULT

The lab demonstrates a valid login check and a failed wrong-password check using the same stored salt and hash. No production DeRexi authentication code or real credentials were modified.

Classroom note: the script uses a reduced PBKDF2 iteration count for lab speed; production settings should follow current organizational guidance.

DEREXI: POLICY PILOT

Message & File Integrity

HMAC-SHA256 • Tamper Detection • SHA-256 File Verification

HMAC MESSAGE INTEGRITY

The lab creates a keyed HMAC for a fictional policy message. The original message verifies successfully; changing “must report” to “may ignore” produces a different tag and is detected.

```
203 # constant-time, so an attacker cannot learn the right value by
204 # timing how long a comparison takes.
205 original_tag = hmac.new(fake_secret_key, policy_message.encode(), hashlib.sha256).digest()
206 original_verified = hmac.compare_digest(tag.hexdigest(), original_tag.hex())
207
208 if original_verified:
209     print("Original message verification: PASS")
210 else:
211     print("Original message verification: FAIL")
212 print("-> The message and its tag match, so nothing was altered.")
213 print()
214
215 # --- Verify a TAMPERED message ---
216 # Change one word ("must report" -> "may ignore") to simulate an
217 # attacker editing the message in transit.
218 tampered_message = (
219     "Aurum Capital Bank employees may ignore suspicious customer "
220     "activity to the BSA/AML Compliance Officer."
221 )
222 tampered_tag = hmac.new(fake_secret_key, tampered_message.encode(), hashlib.sha256).digest()
223 tampered_verified = hmac.compare_digest(tampered_tag.hex(), tag.hexdigest())
```

Figure 2. Original HMAC passes; tampered message is rejected.

RESULTS

Original message: PASS
Modified message: CHANGE DETECTED

Security meaning: HMAC provides integrity plus authenticity because a valid tag requires the shared secret key.

SHA-256 FILE INTEGRITY

A fictional policy file is hashed, verified unchanged, deliberately edited, hashed again, and then restored. Because even a small content change alters the digest, the script can detect accidental or unauthorized modification

```
276 "Policy ID: ACB-SAMPLE-001\n"
277 "Owner: BSA/AML Compliance Officer\n"
278 "\n"
279 "1. Employees must escalate any suspicious customer transaction to\n"
280 "   the BSA/AML Compliance Officer within one business day.\n"
281 "2. Suspicious activity must never be discussed with the customer.\n"
282 "3. All escalations must be recorded in the compliance case log.\n"
283 )
284
285 # --- Step 1: create the sample policy file ---
286 policy_file.write_text(original_contents, encoding="utf-8")
287 print("Created sample file:", policy_file.name)
288 print()
289
290 # --- Step 2: hash the original file and SAVE that hash in memory ---
291 # Saving the original hash first is what makes later comparison possible.
292 original_file_hash = sha256_file(policy_file)
293 print("ORIGINAL FILE HASH (saved in memory):")
294 print(original_file_hash)
295 print()
296
297 # --- Step 3: verify the UNCHANGED file ---
```

Figure 3. A one-word policy change changes the SHA-256 digest.

TEST OUTCOME

Unchanged policy file: PASS
Modified policy file: CHANGE DETECTED
Restored file: PASS

This establishes a repeatable integrity check for policy content.

RUBRIC COVERAGE

Password hashing	PBKDF2-HMAC-SHA256 + random salt	MET
Message integrity	HMAC-SHA256 verification and tamper test	MET
File integrity	SHA-256 digest comparison before/after change	MET